



NATIONAL UNIVERSITY
of Computer & Emerging Sciences

Name: Muhammad Musawar Ali Shah

Roll No. 24P-0619

Dept: BS-CS

Section: BCS-3D

Subject: COAL LAB

Submitted To: Sir Hasaan Shaukat

Lab 6

Question 1:

Code	Explanation
<pre> [org 0x0100] mov bx, 0 mov byte [swap], 0 mov word [count], 0 outer_loop: mov bx, 0 mov byte [swap], 0 inner_loop: mov ax, [data + bx] cmp ax, [data + bx + 2] jae noswap mov dx, [data + bx + 2] mov [data + bx + 2], ax mov [data + bx], dx mov byte [swap], 1 mov ax, [count] inc ax mov [count], ax noswap: add bx, 2 cmp bx, 18 jne inner_loop cmp byte [swap], 1 </pre>	▶ Stored the value 0 in bx register. ▶ Clear swap flag. ▶ Count = 0. ▶ Label outer loop: ▶ in outer loop bx = 0. ▶ reset swap to 0. ▶ In inner loop. Move first value from data into ax register. ▶ Compare value present in data with next one. ▶ If AX value is greater than next value then jump to noswap. ▶ Load next value into dx register (dx working as temp in cpp). ▶ Move the next data into ax register. ▶ Store the value in dx into current slot ▶ move swap = 1 so that we should know that swaping occurred ▶ move value of ax into count. ▶ Increase ax by 1 and then value of ax into count. ▶ If swapping not occurred then: • add 2 into bx. • Check if this is last value or not.

<pre> je outer_loop mov ax, 0x4c00 int 0x21 data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0 swap: db 0 count: dw 0 </pre>	
--	--

Question 2.1:

Code	Explanation
<pre> [org 0x0100] mov ax, 0x00A0 mov bx, ax mov cx, ax shr cx, 8 mov dx, ax shl dx, 8 mov ax, bx shl ax, 4 sar ax, 4 mov ax, 0x4c00 int 0x21 </pre>	▶ Stored a random 16-bit number into ax register ▶ copy value of ax into bx. ▶ Move value of ax into cx. ▶ Shift right by 8 --- upper byte now in lower position. ▶ Now move value of ax into dx. ▶ Shift left by 8 --- lower byte now in upper position. ▶ Move left shift by 4 bits ▶ arithmetic right shift by 4 bit

Question 2.2:

Code	Explanation
<pre> [org 0x0100] mov ax, 1 mov cl, 5 </pre>	▶ Move 1 into ax register. ▶ Store 5 in cl register ▶ shift 1 to the left of cl bits ▶ shift right by cl bits

shl ax, cl shr ax, cl mov ax, 0x4c00 int 0x21	
--	--

Question 3:

Code	Explanation
[org 0x0100] mov bx, 0 mov byte [swap], 0 sort_loop: mov bx, 0 mov byte [swap], 0 inner_sort: mov ax, [data + bx] cmp ax, [data + bx + 2] jbe noswap mov dx, [data + bx + 2] mov [data + bx + 2], ax mov [data + bx], dx mov byte [swap], 1 noswap: add bx, 2	1. Store value 0 in bx register. 2. Initialize 0 in swap register. 3. In outer loop. 4. Do the same thing initialize bx register 0 and swap label 0. 5. in inner loop move first value in ax register. 6. And compare current value with next value. 7. Donot swap if $ax \leq next$ and if $ax > swap$ then swap values. 8. Move next value in dx. 9. Put current element in next slot 10. put saved next element ito current slot 11. make that a swap happened this pass. 12. Add 2 into bx so that we an move to next value 13. compare if we reached the last value. 14. Reset bx to 0. 15. cx = 0. 16. load cutrent sorted element into ax 17. shift left by value in cl 18. logical right shift by 3

<pre> cmp bx, 18 jne inner_sort cmp byte [swap], 1 je sort_loop mov bx, 0 mov cx, 0 shift_loop: mov ax, [data + bx] shl ax, cl shr ax, 3 sar ax, 1 add bx, 2 inc cx cmp bx, 20 jne shift_loop mov ax, 0x4c00 int 0x21 data: dw 60, 55, 45, 50, 40, 35, 25, 30, 10, 0 swap: db 0 </pre>	<pre> 19. sar ax, 1 (arithmetic right by 1 20. add 2 int o bx register. 21. increase value of cx by 1 22. check if we reached last value or not. </pre>
---	---